

The background features a dark gray grid pattern. In the top right and bottom left corners, there are wavy, glowing purple lines that create a sense of motion or energy.

iTMO

Retrieval

О чем будет лекция

Retrieval / Recall / отбор кандидатов / кандидатогенерация / кандген – первый этап воронки, он задает верхнюю границу качества рекомендательной системы

Задача этапа – **быстро** оценить, релевантен ли пользователю объект (а на вопрос «насколько» отвечает уже этап ранжирования)

В одной системе может быть несколько разных кандгенов

Основная метрика – Recall@много



іТМО

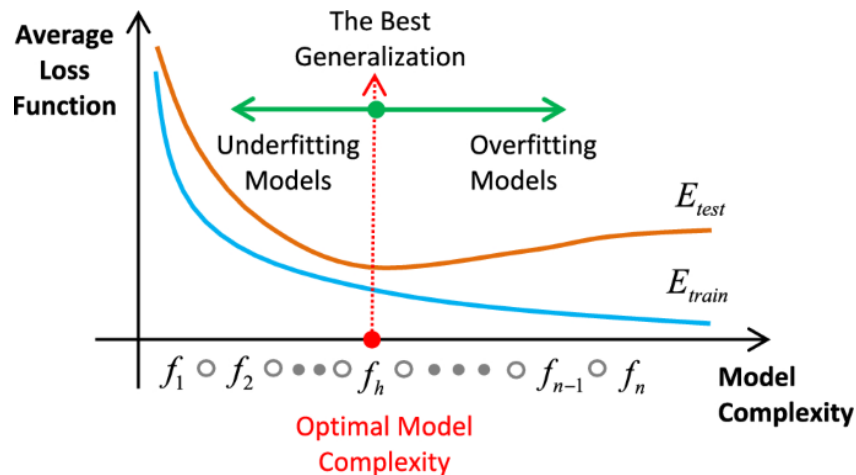
Моделі

Важная оговорка

Все модели **всегда** балансируют между memorization и generalization

Где-то в сферическом вакууме есть идеальная генеральная совокупность, а у нас есть наблюдаемая по ней зашумленная выборка, и нужно на основе этой выборки постараться описать генеральную совокупность. Слабо изучим выборку – не найдем описательные признаки, выучим выборку наизусть – не сможем экстраполироваться

Отсюда вытекают все шаманства с train-val-test

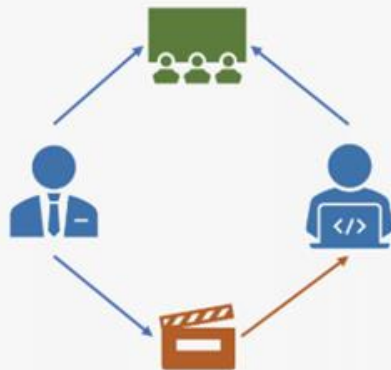


Классификация подходов

Идейно – можем опираться на информацию об интеракциях (коллаборативную) и/или на метаданные айтеров (контент)



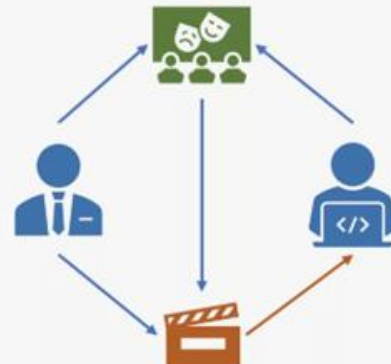
Collaborative Filtering



Content-based Filtering

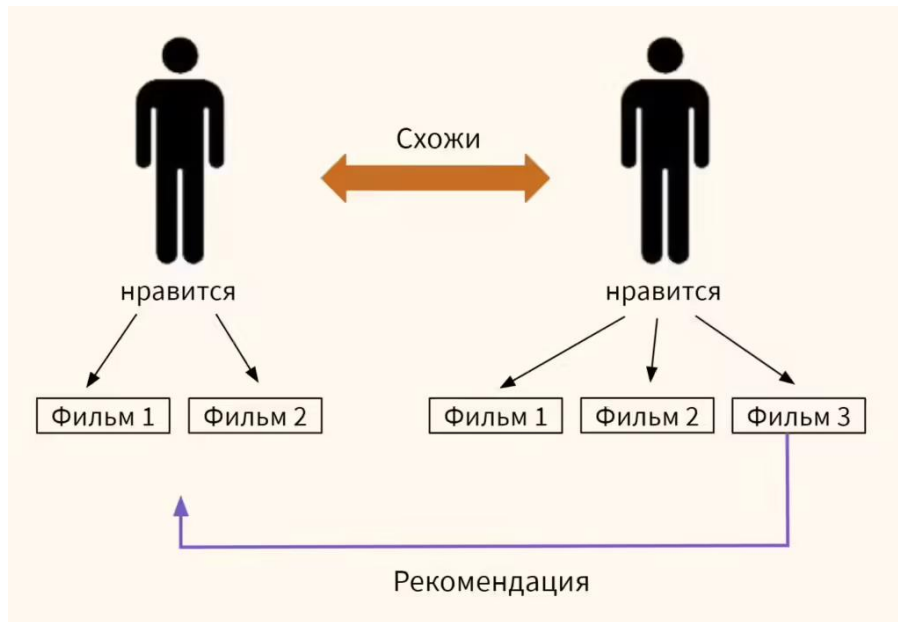


Hybrid systems



Коллаборативная фильтрация

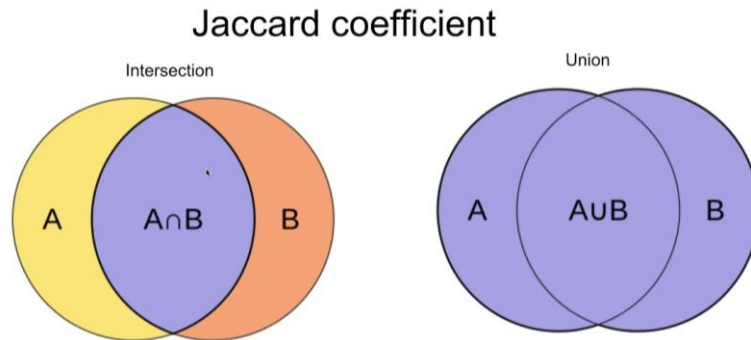
itmo



Коллаборативная фильтрация. kNN

іТМО

1. Пользователь – строка в матрице взаимодействий (фиксируем срез)
2. Для каждого пользователя, которому мы хотим сделать рекомендации, находим наиболее похожих (например, по расстоянию Жаккара) на него пользователей
3. Набираем у соседей айтемы, с которыми наш целевой пользователь еще не взаимодействовал



$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Коллаборативная фильтрация.

Матричная факторизация

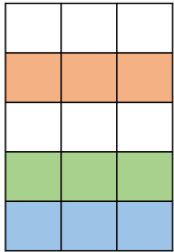
$$\hat{R} = QP^T \quad \hat{r}_{ui} = q_u^T p_i$$



	10	-1	8	10	9	4
	8	9	10	-1	-1	8
	10	5	4	9	-1	-1
	9	10	-1	-1	-1	3
	6	-1	-1	-1	8	10

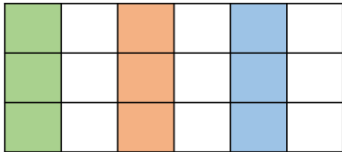
User-item Interaction Matrix
(R)

$$Q \in \mathbb{R}^{|U| \times k}$$



User Matrix
(Q)

$$P \in \mathbb{R}^{|I| \times k}$$



Item Matrix
(P)

Коллаборативная фильтрация. Матричная факторизация. SVD

$$\begin{bmatrix} \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \end{bmatrix} = \begin{bmatrix} \bullet & \color{red}{\bullet} & \bullet & \bullet \\ \bullet & \color{red}{\bullet} & \bullet & \bullet \\ \bullet & \color{red}{\bullet} & \bullet & \bullet \\ \bullet & \color{red}{\bullet} & \bullet & \bullet \end{bmatrix} \begin{bmatrix} \bullet & \color{blue}{\bullet} & \bullet \\ \bullet & \bullet & \bullet \end{bmatrix} \begin{bmatrix} \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \color{green}{\bullet} & \color{green}{\bullet} & \color{green}{\bullet} & \color{green}{\bullet} \\ \bullet & \bullet & \bullet & \bullet & \bullet \end{bmatrix}$$

M
 $n \times m$

U
 $n \times k$
columns are orthonormal

D
 $k \times k$,
 $k = \text{rank } M$
diagonal matrix

V^T
 $k \times m$
rows are orthonormal



Коллаборативная фильтрация.

Матричная факторизация. BPR

$$\mathcal{L}_{BPR} = - \sum_{(u,i,j) \in D} \ln \sigma(\hat{r}_{ui} - \hat{r}_{uj}) + \lambda \|\Theta\|_2^2$$



Для каждого триплета (u, i, j) :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- u — пользователь;
- i — **положительный** item, с которым пользователь взаимодействовал;
- j — **отрицательный** item, с которым пользователь не взаимодействовал;
- $\hat{r}_{ui}$ — предсказанный скор пользователя u для объекта i ;
- $\hat{r}_{uj}$ — скор для отрицательного объекта j ;
- $\lambda \|\Theta\|_2^2$ — L2-регуляризация параметров модели.

$$\hat{r}_{ui} = q_u^\top p_i \quad \mathcal{L}_{BPR} = - \ln \sigma(q_u^\top p_i - q_u^\top p_j) + \lambda (\|q_u\|^2 + \|p_i\|^2 + \|p_j\|^2)$$

Коллаборативная фильтрация. Матричная факторизация. ALS



$$\mathcal{L}_{ALS} = \sum_{(u,i) \in \Omega} (r_{ui} - q_u^\top p_i)^2 + \lambda \left(\sum_u \|q_u\|_2^2 + \sum_i \|p_i\|_2^2 \right)$$

Параметры оптимизируются попеременно: сначала фиксируем все P и решаем least squares для q_u , затем фиксируем Q и обновляем p_i

Для одного шага оптимизации существует решение в закрытой форме:

$$q_u = (P_u^\top P_u + \lambda I)^{-1} P_u^\top r_u$$

Коллаборативная фильтрация. Матричная факторизация. iALS

ITMO

$$y_{ui} = \begin{cases} 1, & r_{ui} > 0 \\ 0, & r_{ui} = 0 \end{cases} \quad c_{ui} = 1 + \alpha r_{ui}$$

$$\mathcal{L}_{iALS} = \sum_{u,i} c_{ui} (y_{ui} - q_u^\top p_i)^2 + \lambda \left(\sum_u \|q_u\|^2 + \sum_i \|p_i\|^2 \right)$$

$$q_u = (P^\top C_u P + \lambda I)^{-1} P^\top C_u y_u \quad C_u = \text{diag}(c_{u1}, \dots, c_{u|I|})$$

С отвечает за то, во сколько раз ошибка на позитивном айтеме весит больше, чем ошибка на неизвестном айтеме

Коллаборативная фильтрация.

Факторизационные машины. LightFM

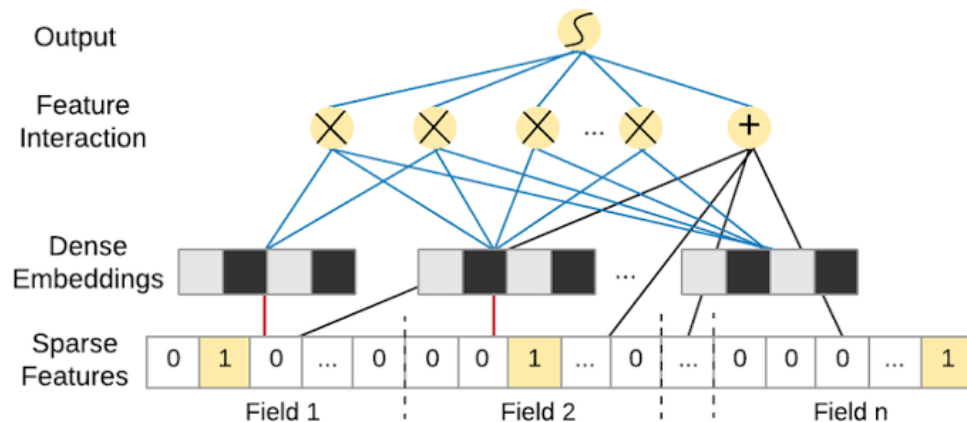
ITMO

- MF: только user_id x item_id
- FM: произвольные sparse-признаки и их попарные взаимодействия
- LightFM (библиотека): user_id/item_id + side features

$$x_{ui} = \left[\underbrace{\text{user features}}_{\text{user id, age, country, ...}}, \underbrace{\text{item features}}_{\text{item id, genre, brand, ...}} \right]$$

$$\hat{y}_{ui} = w_0 + \sum_{j=1}^n w_j x_j + \sum_{j=1}^n \sum_{k=j+1}^n \langle v_j, v_k \rangle x_j x_k$$

Взаимодействия латентных признаков + скейлинг

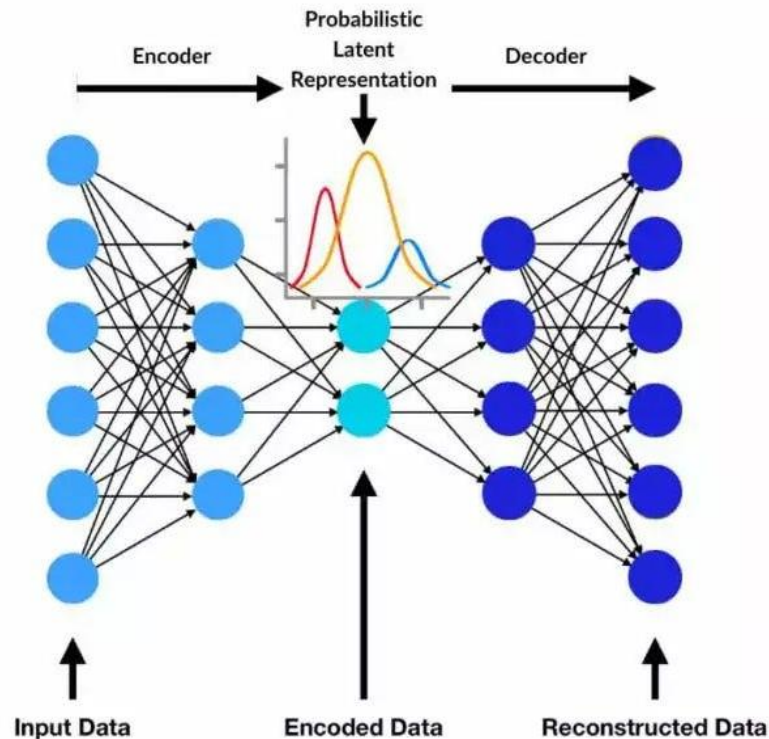


Коллаборативная фильтрация. Автокодировщики. C-VAE

ITMO

Про то, как работает VAE, вам должны рассказывать на курсе DL

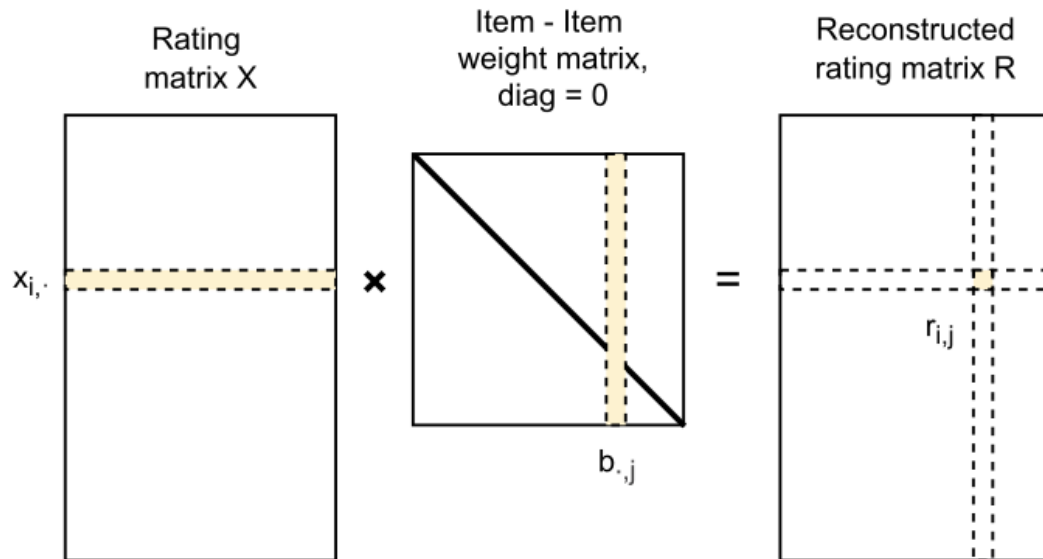
С точки зрения RecSys мы с потерями сжимаем вектор взаимодействий пользователя, и, восстановив его, получаем скоры для айтемов, по которым их можно отранжировать



Линейные модели

Тоже коллаборативные, но информация о юзерах используется только на этапе подготовки обучающих данных

Эти модели формируют I2I матрицу, выражая каждый айтем через все остальные айтемы, кроме него самого



Линейные модели. EASE

$$X \in \mathbb{R}^{|U| \times |I|} \quad B \in \mathbb{R}^{|I| \times |I|} \quad \text{diag}(B) = 0$$



$$X \approx XB$$

$$\mathcal{L}_{EASE} = \|X - XB\|_F^2 + \lambda \|B\|_F^2$$

$$\hat{X} = XB \quad \hat{x}_{uj} = \sum_i x_{ui} B_{ij}$$

Имеет решение в закрытой форме:

$$G = X^\top X \quad G_\lambda = X^\top X + \lambda I \quad P = G_\lambda^{-1}$$

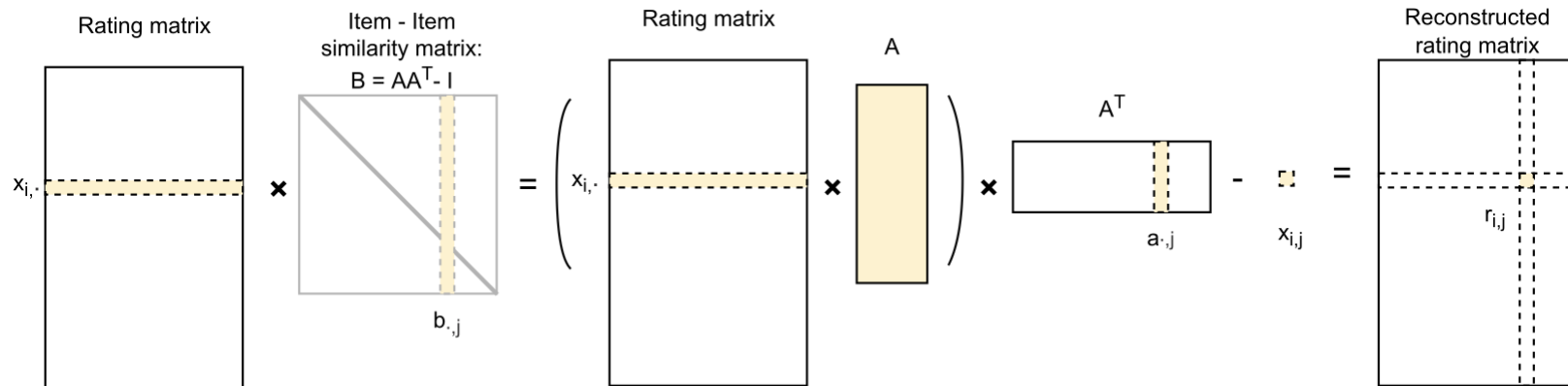
$$B_{ij} = -\frac{P_{ij}}{P_{jj}}, \quad i \neq j$$

$$B_{jj} = 0$$

В матричной форме ->
$$B = -\frac{P}{\text{diag}(P)}$$

Линейные модели. ELSA

Итог похож на EASE, но решение понятнее



Линейные модели. SLIM

Аналог EASE, но добавляем L1, и теперь требуется численная оптимизация



$$X \in \mathbb{R}^{|U| \times |I|} \quad B \in \mathbb{R}^{|I| \times |I|} \quad \text{diag}(B) = 0$$

$$X \approx XB$$

$$\mathcal{L}_{SLIM} = \frac{1}{2} \|X - XB\|_F^2 + \lambda_1 \|B\|_1 + \frac{\lambda_2}{2} \|B\|_F^2 \quad \hat{X} = XB \quad \hat{x}_{uj} = \sum_i x_{ui} B_{ij}$$

$$\min_{\beta_j} \frac{1}{2} \|x_j - X\beta_j\|_2^2 + \lambda_1 \|\beta_j\|_1 + \frac{\lambda_2}{2} \|\beta_j\|_2^2$$

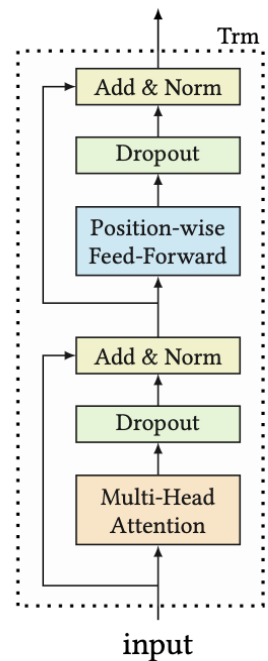
Нейросети

Очень актуально на сейчас

Моделей и методов для кандгена (а сейчас мы говорим про него)
ОЧЕНЬ много, вот какие-то основные классы моделей:

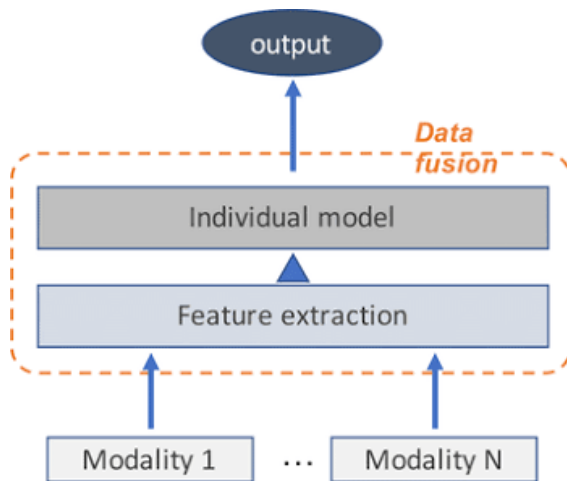
- Two-Tower (сегодня обсудим их)
- LLM-based
- Generative
- Diffusion
- GNN

Что минимально нужно знать тут, чтобы что-то понять: Transformer, Cuasality, Cross-Entropy, Negative Sampling, Contrastive Learning

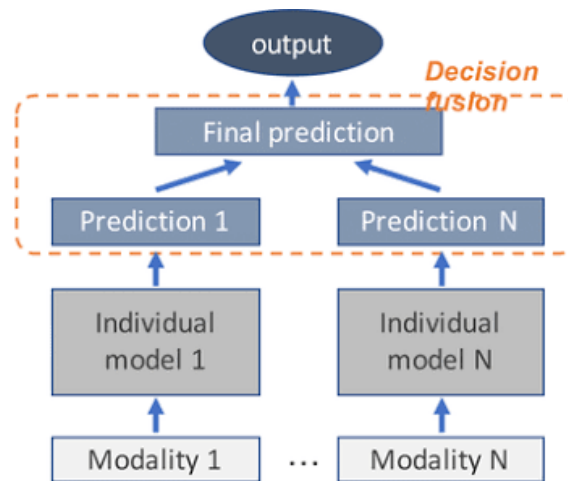


(a) Transformer Layer.

Раннее vs позднее связывание, для кандгена обычно характерно второе (why?)



(a) Early fusion



(b) Late fusion

Нейросети. Two-Tower

Существует понятие двухбашенной модели (Two-Tower)

Такие модели отдельно обрабатывают вход по юзерам и по айтемам, формируя их эмбединги, а затем применяют MIPS (maximum inner product score) для выбора, что порекомендовать

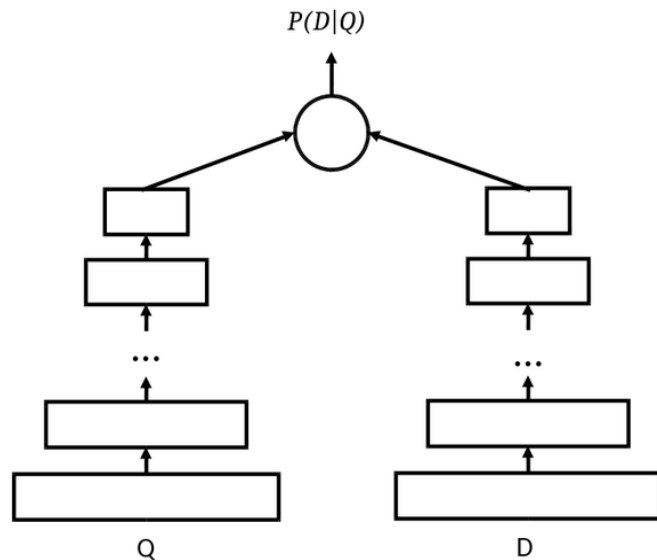
MIPS чаще всего подразумевает dot product, но не обязательно (например, см. статью Retrieval with Learned Similarities)

Эмбединг пользователя формируется по его истории взаимодействий + опционально контексту

Similarity function

Latent semantic
feature vector

Representation
learning



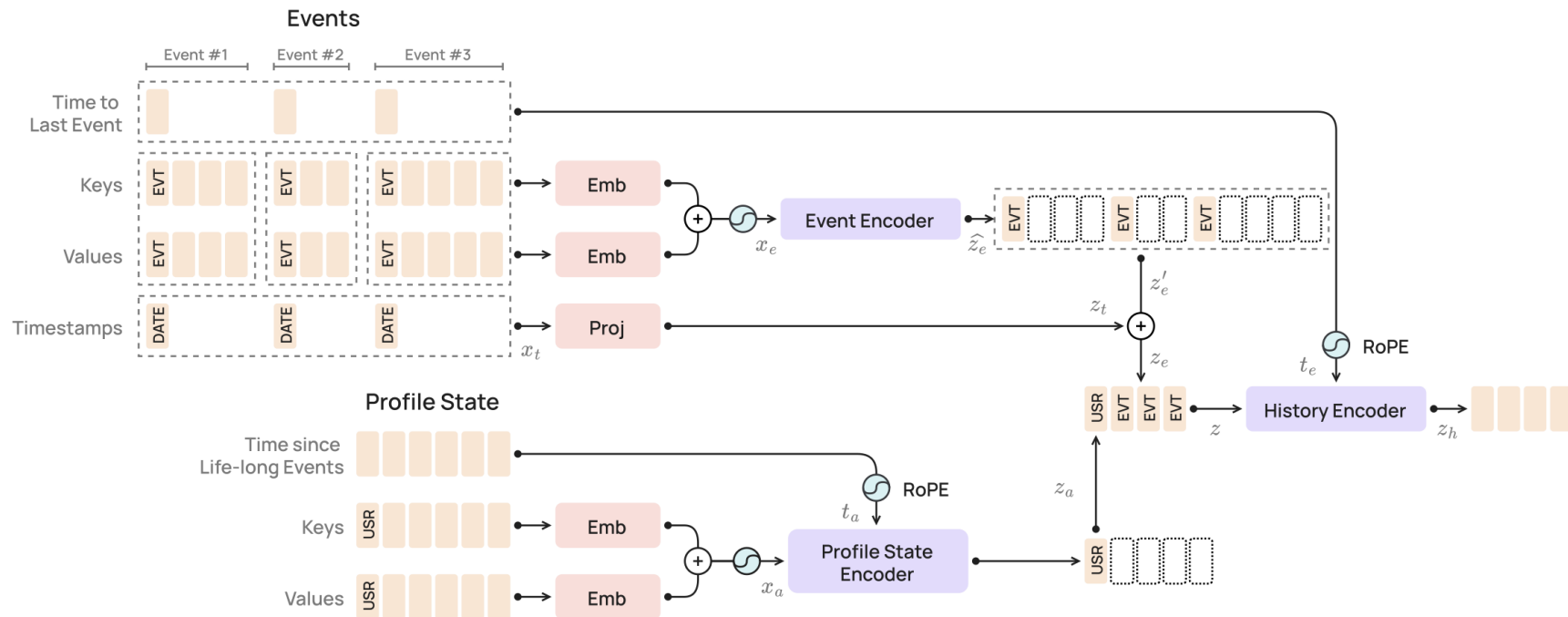
Нейросети. Sequential

Тоже Two-Tower, но айтемная башня тривиальна



Что-то вроде SOTA для кодирования пользователя сейчас

ITMO



Нейросети. Лоссы

$$s_{ui} = q_u^\top p_i$$
ITMO

$$\mathcal{L}_{\text{BCE}} = -y_{ui} \log \sigma(s_{ui}) - (1 - y_{ui}) \log(1 - \sigma(s_{ui}))$$



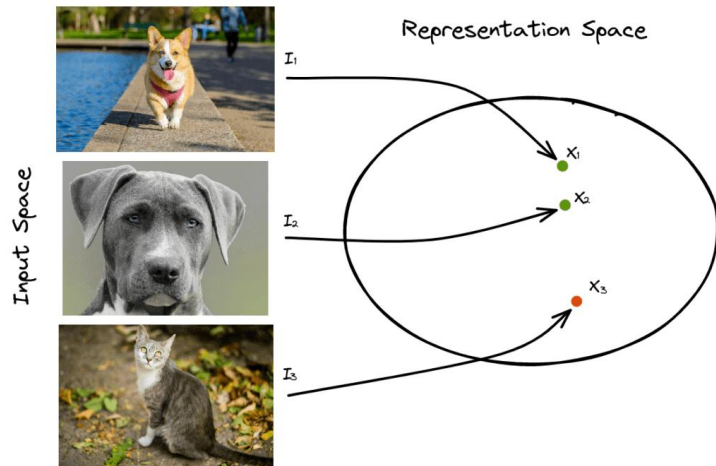
$$\mathcal{L}_{\text{CE}} = -\log \frac{e^{s_{ui}}}{\sum_{k \in I} e^{s_{uk}}}$$

Это всё с точки зрения физического смысла – почти одно и то же! Как в CV:

$$\mathcal{L}_{\text{SampledCE}} = -\log \frac{e^{s_{ui}}}{e^{s_{ui}} + \sum_{j \in N_u} e^{s_{uj}}}$$

$$\mathcal{L}_{\text{Triplet}} = \max(0, m - s_{ui} + s_{uj})$$

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{e^{s_{ui}/\tau}}{e^{s_{ui}/\tau} + \sum_{j \in N_u} e^{s_{uj}/\tau}}$$



Почему вообще кросс-энтропия?

Лирическое отступление в сторону основ DL на всякий случай ☺

Пусть у нас есть набор реальных данных и набор предсказаний модели. В идеале мы хотим, чтобы они совпадали. Если оценим степень несовпадения через дифференцируемую функцию, то сможем её минимизировать. Степень несовпадения можно выразить через KL-дивергенцию:

$$D_{\text{KL}}(P \| Q_{\theta}) = \sum_x P(x) \log \frac{P(x)}{Q_{\theta}(x)}$$

Но чтобы такое дело минимизировать, надо знать $P(x)$, а мы его не знаем. Финт ушами:

$$D_{\text{KL}}(P \| Q_{\theta}) = -H(P) - \sum_x P(x) \log Q_{\theta}(x) \quad H(P) = - \sum_x P(x) \log P(x) \quad \begin{array}{l} \text{Не зависит от} \\ \text{параметров} \\ \text{модели!} \end{array}$$

Отсюда: $\arg \min_{\theta} D_{\text{KL}}(P \| Q_{\theta}) = \arg \min_{\theta} H(P, Q_{\theta})$

$$H(P, Q_{\theta}) = - \sum_x P(x) \log Q_{\theta}(x) \quad H(P, Q_{\theta}) = \mathbb{E}_{x \sim P}[-\log Q_{\theta}(x)] \approx -\frac{1}{n} \sum_{k=1}^n \log Q_{\theta}(x_k)$$

Мы берем реальные события и штрафует модель за низкую вероятность для этих событий

Нейросети. Negative Sampling

Каталог айтемов – большой
FullCE зачастую физически не посчитать,
поэтому используется SampledCE

Откуда взять негативы?
Насэмплировать!

Стратегии сэмплирования:

- Uniform
- Popularity-weighted (тут есть некоторая проблема)

Откуда сэмплировать:

- In-batch
- Cross-batch

1 Too easy

Positive (user likes)



Random negatives (obviously unrelated)



Random negatives teach broad interest

Good for easy separation,
weak for fine-grained ranking.

2 More realistic, but risky

Positive (user likes)



Hard negatives (similar items)



Hard negatives teach fine-grained preference

Useful, but non-click does not
always mean dislike.

3 Practical production setup

Mix strategies to learn multiple skills
and reflect real-world scenarios.

random

same-category

popular

in-batch

exposed non-click

hard negatives

💡 Different negatives teach
different skills.

📊 Track performance by
negative source.



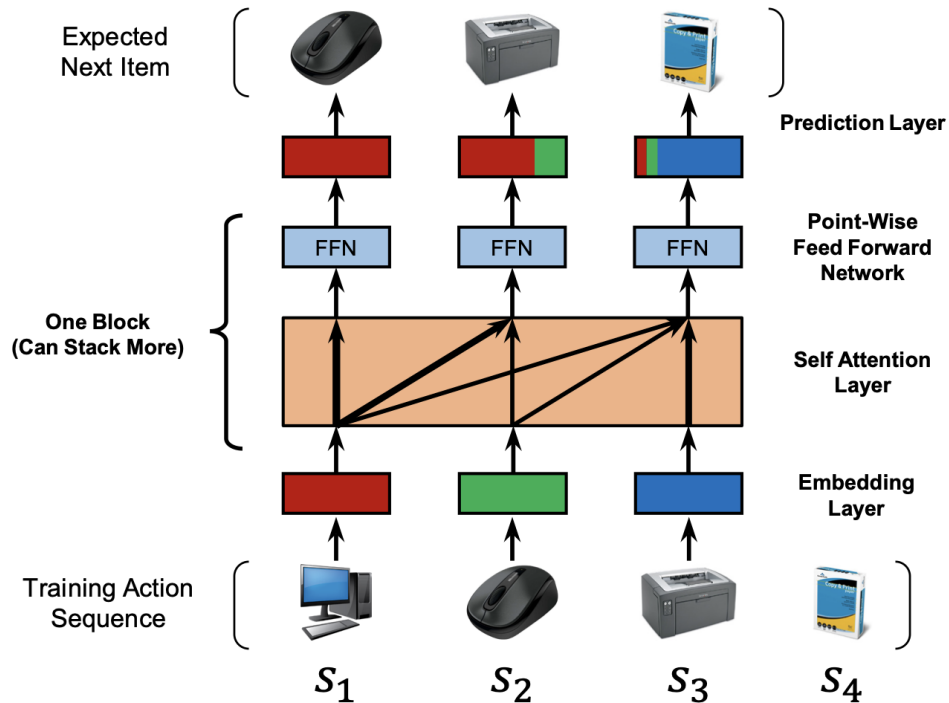
A lower loss can just mean the model is winning on easy negatives.

Нейросети. SASRec

Задача – Next Item Prediction (NTP)

В Vanilla SASRec используется каузальная маска и BCE

У модели есть множество доработок и вариаций, и она всё ещё SOTA

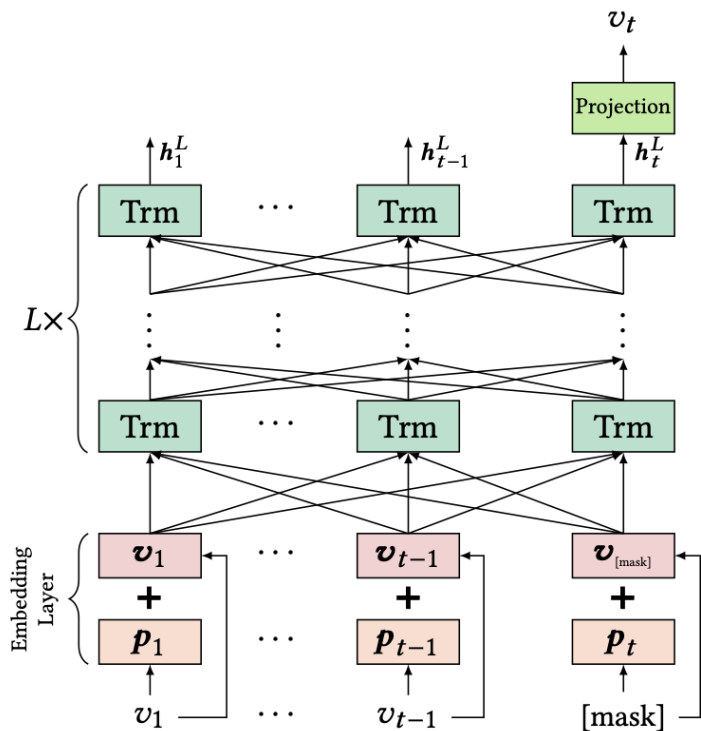


Нейросети. BERT4Rec

Задача – Masked Item Prediction

В Vanilla BERT4Rec используется
двунаправленное внимание и FullCE

Хорошо подходит для претрейна, плохо –
для всего остального. При переходе на
SampledCE проигрывает SASRec-y

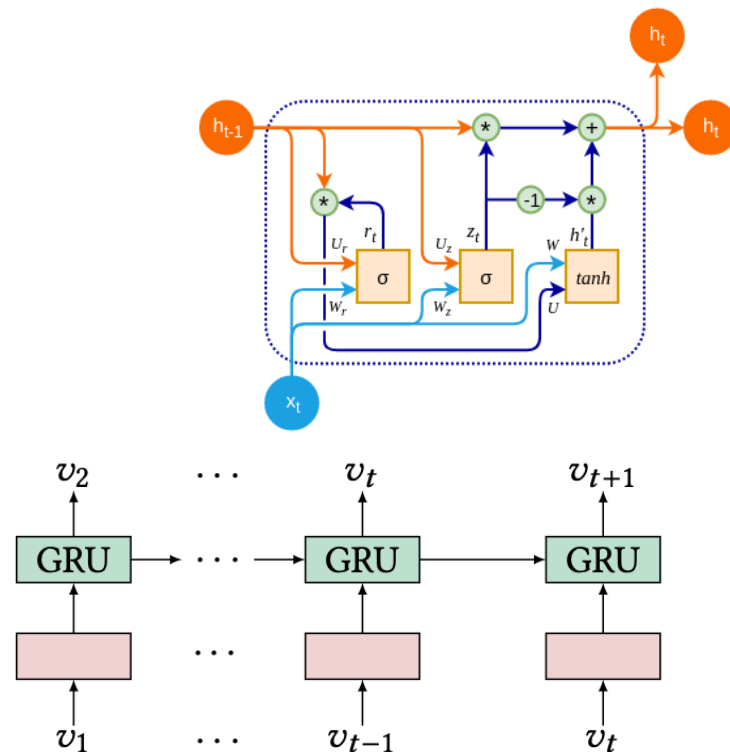


Нейросети. GRU4Rec

Задача – Next Item Prediction

В Vanilla GRU4Rec используется каузальная рекуррентная архитектура на основе GRU и FullCE

Хорошо подходит для step-by-step ранжирования в реальном времени, плохо масштабируется на длинные последовательности



The background is a dark, almost black, grid pattern. Overlaid on this grid are several wavy, glowing purple lines that create a sense of movement and depth. These lines are most prominent in the top right and bottom left corners, where they form complex, looping shapes. The overall aesthetic is modern and digital.

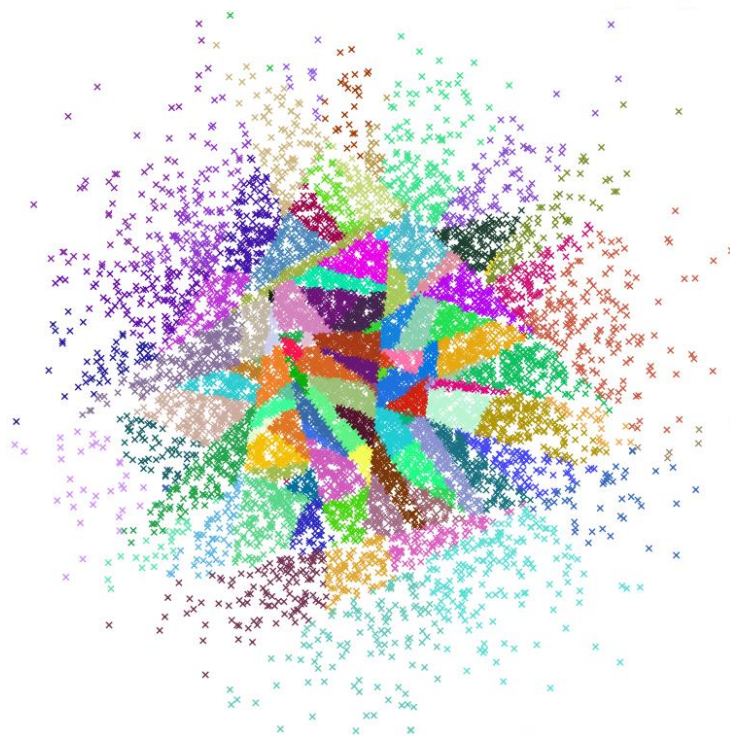
iTMO

ANN

Поисковые индексы. FAISS

Индекс – это структура данных (обычно ANN-индекс, например HNSW или IVF), которая хранит эмбединги айтемов и позволяет быстро находить ближайших соседей к вектору запроса вместо полного перебора

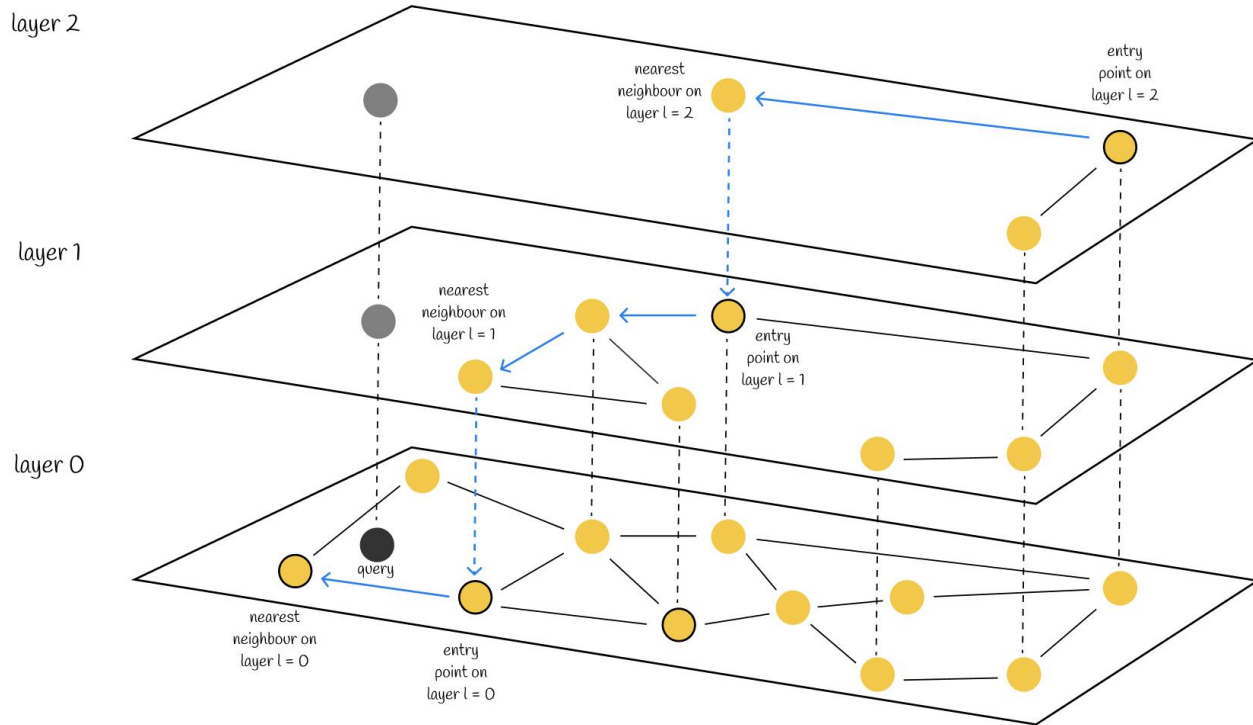
<https://github.com/facebookresearch/faiss>



HNSW

Идейно похоже на
SkipList

Что важно помнить:
такая структура данных
организует эмбединги
не по признаку
семантической
близости, а так, как ей
удобно



Векторные БД

Базы данных, оптимизированные для хранения эмбеддингов и быстрого поиска ближайших соседей по ним, в отличие от обычных БД, работающих с точными значениями и скалярными запросами. Скалярные запросы обычно тоже поддерживают



Vertex AI



Qdrant



PostgreSQL



pgvector



weaviate



Chroma



PINECONE

Спасибо за внимание!

itMO *re than a*
UNIVERSITY

334743@niuitmo.ru